SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA0201

COURSE NAME: C Programming

COURSE OUTCOME COVERED

CO2: SCENARIO-BASED ASSESSMENTS

. (BL2,BL3)

Assessment Tool Weightage: 20%

QUESTIONS: Total Marks:100

Annexure A

EXPERIMENTS

Code of Conduct:

I, ASHIRA BRIDGET SUSAN S (192524316) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Q1.

1. Electric Vehicle Battery Manufacturing System

An electric vehicle battery manufacturing company produced batteries from January 2026 to December 2026. Each month, the target production is 5,000 batteries. If production exceeds the target, the company receives a 10% performance incentive. If production is below 80% of the target, the production unit is marked for inspection.

(a)

Develop a C program using arithmetic operators to calculate the annual production, average monthly production, and production efficiency. Use decision-making statements to determine the incentive and inspection status.

(b)

Use looping constructs to process production details for all 12 months. Use continue to skip months with invalid production values and break if the plant shuts down due to equipment failure.

Pseudocode:

```
START
SET TARGET = 5000
SET totalProduction = 0
FOR month = 1 TO 12
   INPUT production
   IF production = -1 THEN
      PRINT "Plant shut down."
      BREAK
   ENDIF
   IF production < 0 THEN
      PRINT "Invalid production value."
      CONTINUE
   ENDIF
   totalProduction = totalProduction + production
   IF production > TARGET THEN
      PRINT "10% Performance Incentive Awarded."
   ENDIF
   IF production < (0.8 × TARGET) THEN
      PRINT "Production Unit Marked for Inspection."
   ENDIF
   efficiency = (production × 100) / TARGET
   PRINT efficiency
END FOR
```

averageProduction = totalProduction / 12

annualEfficiency = (averageProduction × 100) / TARGET

PRINT totalProduction

PRINT averageProduction

PRINT annualEfficiency

STOP

C Pogram:

```
#include <stdio.h
#define TARGET 5000
#define MONTHS 12
int main()
{
   int production, i;
   int totalProduction = 0;
   float averageProduction, efficiency;
   for(i = 1; i <= MONTHS; i++)
   {
      printf("\nMonth %d\n", i);
      printf("Enter batteries produced: ");
      scanf("%d", &production);
      if(production == -1)
      {
         printf("Plant shut down due to equipment failure.\n");
         break;
      }
      if(production < 0)
      {
         printf("Invalid production value! Skipping this month.\n");
         continue;
      }
      totalProduction += production;
      if(production > TARGET)
```

```
        {
            printf("10%% Performance Incentive Awarded.\n");
        }
        if(production < (0.8 * TARGET))
        {
            printf("Production Unit Marked for Inspection.\n");
        }
        efficiency = ((float)production / TARGET) * 100;
        printf("Production Efficiency = %.2f%%\n", efficiency);
    }
    averageProduction = (float)totalProduction / MONTHS;
    printf("\n========== Annual Report ==========\n");
    printf("Total Annual Production = %d batteries\n", totalProduction);
    printf("Average Monthly Production = %.2f batteries\n", averageProduction);
    efficiency = ((float)averageProduction / TARGET) * 100;
    printf("Average Production Efficiency = %.2f%%\n", efficiency);
    return 0;
}
```

Output:

```
Output

Month 1
Enter batteries produced: 5200
10% Performance Incentive Awarded.
Production Efficiency = 104.00%

Month 2
Enter batteries produced: 4800
Production Efficiency = 96.00%

Month 3
Enter batteries produced: 6000
10% Performance Incentive Awarded.
Production Efficiency = 120.00%

Month 4
Enter batteries produced: 3900
Production Unit Marked for Inspection.
Production Efficiency = 78.00%

Month 5
Enter batteries produced: 5100
10% Performance Incentive Awarded.
Production Efficiency = 102.00%

Month 6
Enter batteries produced: 5300
10% Performance Incentive Awarded.
Production Efficiency = 106.00%

Month 7
Enter batteries produced: 4700
Production Efficiency = 94.00%

Month 8
Enter batteries produced: 5500
10% Performance Incentive Awarded.
Production Efficiency = 110.00%
```

```
Output

Enter batteries produced: 5300
10% Performance Incentive Awarded.
Production Efficiency = 106.00%

Month 7
Enter batteries produced: 4700
Production Efficiency = 94.00%

Month 8
Enter batteries produced: 5500
10% Performance Incentive Awarded.
Production Efficiency = 110.00%

Month 9
Enter batteries produced: 6200
10% Performance Incentive Awarded.
Production Efficiency = 124.00%

Month 10
Enter batteries produced: 4900
Production Efficiency = 98.00%

Month 11
Enter batteries produced: 4500
Production Efficiency = 90.00%

Month 12
Enter batteries produced: 5800
10% Performance Incentive Awarded.
Production Efficiency = 116.00%

========== Annual Report ==========
Total Annual Production = 61900 batteries
Average Monthly Production = 5158.33 batteries
Average Production Efficiency = 103.17%


=== Code Execution Successful ===
```

Q2.

2. IT Company Project Performance Evaluation (2025–2026)
An IT company completed 20 software projects during the financial year 2025–2026. Each project is evaluated based on Quality Score (100 marks) and Delivery Delay (days).
Performance Criteria:
Quality ≥ 90 and Delay ≤ 2 days → Excellent
Quality 75–89 and Delay ≤ 5 days → Good
Quality 60–74 → Average
Below 60 → Needs Improvement
(a)
Develop a C program using arithmetic operators to calculate the average quality score and delivery percentage. Use decision-making statements to classify each project.
(b)
Use looping constructs to process all 20 projects. Use continue to ignore cancelled projects and break if management stops the evaluation.

Pseudocode:

```
START
SET totalQuality = 0
SET totalDelay = 0
FOR project = 1 TO 20
   INPUT qualityScore
   IF qualityScore = -1 THEN
      PRINT "Project Cancelled."
      CONTINUE
   ENDIF
   INPUT deliveryDelay
   IF deliveryDelay = -99 THEN
      PRINT "Management stopped evaluation."
      BREAK
   ENDIF
   totalQuality = totalQuality + qualityScore
   totalDelay = totalDelay + deliveryDelay
   IF qualityScore >= 90 AND deliveryDelay <= 2 THEN
      PRINT "Excellent"
   ELSE IF qualityScore >= 75 AND
         qualityScore <= 89 AND
         deliveryDelay <= 5 THEN
      PRINT "Good"
```

```
    ELSE IF qualityScore >= 60 AND
            qualityScore <= 74 THEN
        PRINT "Average"
    ELSE
        PRINT "Needs Improvement"
    ENDIF
END FOR
averageQuality = totalQuality / 20
deliveryPercentage =
((5 - (totalDelay / 20)) × 100) / 5
PRINT averageQuality
PRINT deliveryPercentage
STOP
```

C Program:

```
#include <stdio.h>
#define PROJECTS 20
int main()
{
    int i;
    float quality, delay;
    float totalQuality = 0, totalDelay = 0;
    float avgQuality, deliveryPercentage;
    for(i = 1; i <= PROJECTS; i++)
    {
        printf("\nProject %d\n", i);
        printf("Enter Quality Score (Enter -1 if cancelled): ");
        scanf("%f", &quality);
        if(quality == -1)
        {
            printf("Project Cancelled! Skipping.\n");
            continue;
```

```
    }
    printf("Enter Delivery Delay (days): ");
    scanf("%f", &delay);

    // Management stops evaluation
    if(delay == -99)
    {
        printf("Management stopped the evaluation.\n");
        break;
    }
    totalQuality += quality;
    totalDelay += delay;
    // Project Classification
    if(quality >= 90 && delay <= 2)
    {
        printf("Performance : Excellent\n");
    }
    else if(quality >= 75 && quality <= 89 && delay <= 5)
    {
        printf("Performance : Good\n");
    }
    else if(quality >= 60 && quality <= 74)
    {
        printf("Performance : Average\n");
    }
    else
    {
        printf("Performance : Needs Improvement\n");
    }
}
avgQuality = totalQuality / PROJECTS;
```

```
    /* Assuming maximum acceptable delay = 5 days */
    deliveryPercentage = ((5 - (totalDelay / PROJECTS)) / 5) * 100;
    if(deliveryPercentage < 0)
        deliveryPercentage = 0;
    printf("\n========== Performance Report ==========\n");
    printf("Average Quality Score = %.2f\n", avgQuality);
    printf("Delivery Percentage = %.2f%%\n", deliveryPercentage);

    return 0;
}
```

Output

```
Project 1
Enter Quality Score (Enter -1 if cancelled): 95
Enter Delivery Delay (days): 2
Performance : Excellent

Project 2
Enter Quality Score (Enter -1 if cancelled): 88
Enter Delivery Delay (days): 3
Performance : Good

Project 3
Enter Quality Score (Enter -1 if cancelled): 80
Enter Delivery Delay (days): -99
Management stopped the evaluation.

========== Performance Report ==========
Average Quality Score = 9.15
Delivery Percentage = 95.00%
```

Q3.

3. Smart Grid Power Distribution System
A power distribution company monitors 15 substations every hour. Each substation has a maximum capacity of 500 MW.
Business Rules:
Load ≤ 60% → Normal
61–85% → High Load
Above 85% → Critical
Above 500 MW → Grid Failure
(a)
Develop a C program using arithmetic operators to calculate the load percentage and available capacity. Use decision-making statements to classify each substation.
(b)
Use looping constructs to monitor all substations. Use continue to skip substations under maintenance and break if grid failure occurs.

Pseudocode:

```
START
SET MAX_CAPACITY = 500
SET totalLoad = 0
FOR substation = 1 TO 15
   INPUT load
   IF load = -1 THEN
      PRINT "Substation Under Maintenance."
      CONTINUE
   ENDIF
   IF load > 500 THEN
      PRINT "GRID FAILURE OCCURRED!"
      BREAK
   ENDIF
   totalLoad = totalLoad + load
   loadPercentage = (load × 100)/500
   availableCapacity = 500 - load
   IF loadPercentage <= 60 THEN
      PRINT "Status : Normal"
   ELSE IF loadPercentage <= 85 THEN
      PRINT "Status : High Load"
   ELSE
      PRINT "Status : Critical"
   ENDIF
```

```
    PRINT loadPercentage
    PRINT availableCapacity
END FOR
PRINT totalLoad
STOP
```

C Program:

```
#include <stdio.h>

#define MAX_CAPACITY 500
#define SUBSTATIONS 15

int main()
{
    int i;
    float load, totalLoad = 0;
    float loadPercentage, availableCapacity;

    for(i = 1; i <= SUBSTATIONS; i++)
    {
        printf("\nSubstation %d\n", i);
        printf("Enter Current Load (MW): ");
        scanf("%f", &load);

        // Under maintenance
        if(load == -1)
        {
            printf("Substation Under Maintenance! Skipping...\n");
            continue;
        }

        // Grid failure
```

```
    if(load > MAX_CAPACITY)
    {
        printf("GRID FAILURE OCCURRED!\n");
        break;
    }

    totalLoad += load;

    loadPercentage = (load * 100) / MAX_CAPACITY;
    availableCapacity = MAX_CAPACITY - load;

    printf("Load Percentage = %.2f%%\n", loadPercentage);
    printf("Available Capacity = %.2f MW\n",
            availableCapacity);

    // Classification
    if(loadPercentage <= 60)
    {
        printf("Status : Normal\n");
    }
    else if(loadPercentage <= 85)
    {
        printf("Status : High Load\n");
    }
    else
    {
        printf("Status : Critical\n");
    }
}

printf("\n========== Smart Grid Report ==========\n");
printf("Total Load = %.2f MW\n", totalLoad);
```

```
    return 0;
}
```

Output:

main.c | Output

```
Substation 1
Enter Current Load (MW): 300
Load Percentage = 60.00%
Available Capacity = 200.00 MW
Status : Normal

Substation 2
Enter Current Load (MW): -1
Substation Under Maintenance! Skipping...

Substation 3
Enter Current Load (MW): 450
Load Percentage = 90.00%
Available Capacity = 50.00 MW
Status : Critical

========== Smart Grid Report ==========
Total Load = 4600.00 MW


=== Code Execution Successful ===
```

Q4.

4. Pharmaceutical Vaccine Distribution System
A pharmaceutical company manufactured 2,50,000 vaccine doses in 2026. Vaccines are distributed to 25 hospitals. Each hospital must receive at least 8,000 doses.
Distribution Status:
≥10,000 doses → Sufficient
8,000–9,999 → Minimum Requirement
Below 8,000 → Emergency Supply Required
(a)
Develop a C program using arithmetic operators to calculate remaining vaccine stock after distribution. Use decision-making statements to classify each hospital.
(b)
Use looping constructs to process all 25 hospitals. Use continue for closed hospitals and break if stock becomes zero.

Pseudocode:

```
START

SET TOTAL_STOCK = 250000
SET totalDistributed = 0

FOR hospital = 1 TO 25

   INPUT dosesDistributed

   IF dosesDistributed = -1 THEN
      PRINT "Hospital Closed."
      CONTINUE
   ENDIF

   IF TOTAL_STOCK = 0 THEN
      PRINT "Vaccine Stock Exhausted."
      BREAK
   ENDIF

   IF dosesDistributed > TOTAL_STOCK THEN
      PRINT "Insufficient Stock!"
      BREAK
   ENDIF
```

```
    TOTAL_STOCK = TOTAL_STOCK - dosesDistributed
    totalDistributed = totalDistributed + dosesDistributed

    IF dosesDistributed >= 10000 THEN
        PRINT "Status : Sufficient"

    ELSE IF dosesDistributed >= 8000 THEN
        PRINT "Status : Minimum Requirement"

    ELSE
        PRINT "Status : Emergency Supply Required"
    ENDIF

    PRINT Remaining Stock

END FOR

PRINT Total Distributed
PRINT Remaining Vaccine Stock

STOP
```

C PROGRAM:

```
#include <stdio.h>

#define TOTAL_STOCK 250000
#define HOSPITALS 25

int main()
{
    int i;
```

```
int doses;
int remainingStock = TOTAL_STOCK;
int totalDistributed = 0;

for(i = 1; i <= HOSPITALS; i++)
{
   printf("\nHospital %d\n", i);
   printf("Enter Number of Vaccine Doses Distributed: ");
   scanf("%d", &doses);

   // Closed hospital
   if(doses == -1)
   {
      printf("Hospital Closed! Skipping...\n");
      continue;
   }

   // Stock exhausted
   if(remainingStock == 0)
   {
      printf("Vaccine Stock Exhausted!\n");
      break;
   }

   // Insufficient stock
   if(doses > remainingStock)
   {
      printf("Insufficient Vaccine Stock!\n");
      break;
   }

   totalDistributed += doses;
```

```
        remainingStock -= doses;

        // Classification
        if(doses >= 10000)
        {
            printf("Status : Sufficient\n");
        }
        else if(doses >= 8000)
        {
            printf("Status : Minimum Requirement\n");
        }
        else
        {
            printf("Status : Emergency Supply Required\n");
        }

        printf("Remaining Stock = %d doses\n",
                remainingStock);
    }

    printf("\n========== Distribution Report ==========\n");
    printf("Total Distributed = %d doses\n",
            totalDistributed);
    printf("Remaining Vaccine Stock = %d doses\n",
            remainingStock);

    return 0;
}
```

OUTPUT:

```
Hospital 1
Enter Number of Vaccine Doses Distributed: 12000
Status : Sufficient
Remaining Stock = 238000 doses

Hospital 2
Enter Number of Vaccine Doses Distributed: 8500
Status : Minimum Requirement
Remaining Stock = 229500 doses

Hospital 3
Enter Number of Vaccine Doses Distributed: 7500
Status : Emergency Supply Required
Remaining Stock = 222000 doses

Hospital 4
Enter Number of Vaccine Doses Distributed: 15000
Status : Sufficient
Remaining Stock = 207000 doses

Hospital 5
Enter Number of Vaccine Doses Distributed: 10000
Status : Sufficient
Remaining Stock = 197000 doses
```

5. Airport Passenger Security Screening System
An international airport processes 1,500 passengers daily.
Security Rules:
Age below 18 → Minor Screening
Age 18–59 → Normal Screening
Age 60 and above → Senior Citizen Screening
Carrying baggage above 30 kg → Additional Inspection
Carrying prohibited items → Entry Denied
(a)
Develop a C program using arithmetic operators to calculate total baggage weight and screening statistics. Use decision-making statements to determine passenger screening status.
(b)
Use looping constructs to process passengers. Use continue to skip cancelled tickets and break during emergency evacuation.

PSEUDOCODE

```
START

SET totalBaggage = 0

FOR passenger = 1 TO 1500

   INPUT age

   IF age = -1 THEN
      PRINT "Ticket Cancelled."
      CONTINUE
   ENDIF

   IF age = 999 THEN
      PRINT "Emergency Evacuation!"
      BREAK
   ENDIF

   INPUT baggageWeight
   INPUT prohibitedItems
```

```
    totalBaggage = totalBaggage + baggageWeight

    IF prohibitedItems = 1 THEN
        PRINT "ENTRY DENIED"

    ELSE

        IF age < 18 THEN
            PRINT "Minor Screening"

        ELSE IF age <= 59 THEN
            PRINT "Normal Screening"

        ELSE
            PRINT "Senior Citizen Screening"

        ENDIF

        IF baggageWeight > 30 THEN
            PRINT "Additional Inspection Required"
        ENDIF

    ENDIF

END FOR

PRINT totalBaggage

STOP
```

C PROGRAM:

```
#include <stdio.h>
```

```
#define PASSENGERS 1500

int main()
{
   int i, age, prohibited;
   float baggageWeight;
   float totalBaggage = 0;

   for(i = 1; i <= PASSENGERS; i++)
   {
      printf("\nPassenger %d\n", i);

      printf("Enter Age: ");
      scanf("%d",&age);

      // Cancelled ticket
      if(age == -1)
      {
         printf("Ticket Cancelled! Skipping...\n");
         continue;
      }

      // Emergency evacuation
      if(age == 999)
      {
         printf("EMERGENCY EVACUATION!\n");
         break;
      }

      printf("Enter Baggage Weight (kg): ");
      scanf("%f",&baggageWeight);
```

```
printf("Carrying Prohibited Items? (1-YES / 0-NO): ");
scanf("%d",&prohibited);

totalBaggage += baggageWeight;

// Entry denied
if(prohibited == 1)
{
   printf("Status : ENTRY DENIED\n");
}
else
{
   // Screening based on age
   if(age < 18)
   {
      printf("Screening : Minor Screening\n");
   }
   else if(age <= 59)
   {
      printf("Screening : Normal Screening\n");
   }
   else
   {
      printf("Screening : Senior Citizen Screening\n");
   }

   // Additional inspection
   if(baggageWeight > 30)
   {
      printf("Additional Inspection Required.\n");
   }
```

```
        }
    }

    printf("\n========== Screening Report ==========\n");
    printf("Total Baggage Weight = %.2f kg\n",
            totalBaggage);


    return 0;
}
```

OUTPUT

```
Passenger 1
Enter Age: 15
Enter Baggage Weight (kg): 18
Carrying Prohibited Items? (1-YES / 0-NO): 0
Screening : Minor Screening

Passenger 2
Enter Age: 22
Enter Baggage Weight (kg): 44
Carrying Prohibited Items? (1-YES / 0-NO): 0
Screening : Normal Screening
Additional Inspection Required.

Passenger 3
Enter Age: 22
Enter Baggage Weight (kg): 21
Carrying Prohibited Items? (1-YES / 0-NO): 0
Screening : Normal Screening

Passenger 4
Enter Age: 12
Enter Baggage Weight (kg): 22
Carrying Prohibited Items? (1-YES / 0-NO): 1
Status : ENTRY DENIED
```

6. Semiconductor Chip Manufacturing System
A semiconductor company manufactures 10,000 chips per day.
Quality Standards:
Defect Rate <1% → Grade A
1–3% → Grade B
3–5% → Grade C
Above 5% → Reject Batch
(a)
Develop a C program using arithmetic operators to calculate the defect percentage and accepted chips. Use decision-making statements to assign quality grades.
(b)
Use looping constructs to process production for 30 days. Use continue for maintenance days and break if a machine breakdown occurs.

PSEUDOCODE

```
START

SET TOTAL_CHIPS = 10000

FOR day = 1 TO 30

   INPUT defectiveChips

   IF defectiveChips = -1 THEN
      PRINT "Maintenance Day."
      CONTINUE
   ENDIF

   IF defectiveChips = 999 THEN
      PRINT "Machine Breakdown!"
      BREAK
   ENDIF

   defectPercentage = (defectiveChips × 100)/10000
   acceptedChips = 10000 - defectiveChips

   IF defectPercentage < 1 THEN
      PRINT "Grade A"
```

```
    ELSE IF defectPercentage <= 3 THEN
        PRINT "Grade B"

    ELSE IF defectPercentage <= 5 THEN
        PRINT "Grade C"

    ELSE
        PRINT "Reject Batch"

    ENDIF

    PRINT acceptedChips

END FOR

STOP
```

C PROGRAM

```
#include <stdio.h>

#define TOTAL_CHIPS 10000
#define DAYS 30

int main()
{
    int i, defectiveChips, acceptedChips;
    float defectPercentage;

    for(i = 1; i <= DAYS; i++)
    {
        printf("\nDay %d\n", i);
        printf("Enter Number of Defective Chips : ");
        scanf("%d", &defectiveChips);
```

```
// Maintenance Day
if(defectiveChips == -1)
{
   printf("Maintenance Day! Skipping...\n");
   continue;
}

// Machine Breakdown
if(defectiveChips == 999)
{
   printf("MACHINE BREAKDOWN OCCURRED!\n");
   break;
}

defectPercentage =
((float)defectiveChips * 100) / TOTAL_CHIPS;

acceptedChips =
TOTAL_CHIPS - defectiveChips;

printf("Defect Percentage = %.2f%%\n",
       defectPercentage);

printf("Accepted Chips = %d\n",
       acceptedChips);

// Grade Allocation
if(defectPercentage < 1)
{
   printf("Quality Grade : Grade A\n");
}
```

```
        else if(defectPercentage <= 3)
        {
            printf("Quality Grade : Grade B\n");
        }
        else if(defectPercentage <= 5)
        {
            printf("Quality Grade : Grade C\n");
        }
        else
        {
            printf("Quality Grade : Reject Batch\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
Day 1
Enter Number of Defective Chips : 50
Defect Percentage = 0.50%
Accepted Chips = 9950
Quality Grade : Grade A

Day 2
Enter Number of Defective Chips : 200
Defect Percentage = 2.00%
Accepted Chips = 9800
Quality Grade : Grade B

Day 3
Enter Number of Defective Chips : 40
Defect Percentage = 0.40%
Accepted Chips = 9960
Quality Grade : Grade A

Day 4
Enter Number of Defective Chips : 700
Defect Percentage = 7.00%
Accepted Chips = 9300
Quality Grade : Reject Batch
```

7. Cloud Storage Subscription Management (2026)
A cloud company offers storage plans:
Basic – 100 GB
Standard – 500 GB
Enterprise – 2 TB
Extra storage costs ₹4 per GB.
If storage usage exceeds 95%, a warning should be generated.
(a)
Develop a C program using arithmetic operators to calculate used storage, remaining storage, and extra charges. Use decision-making statements to determine storage status.
(b)
Use looping constructs to process multiple customers. Use continue for inactive accounts and break if server storage becomes full.

PSEUDOCODE

```
START

FOR each customer

   INPUT planType

   IF planType = -1 THEN
      PRINT "Inactive Account."
      CONTINUE
   ENDIF

   IF planType = 999 THEN
      PRINT "Server Storage Full!"
      BREAK
   ENDIF

   INPUT usedStorage

   IF planType = 1 THEN
      totalStorage = 100

   ELSE IF planType = 2 THEN
      totalStorage = 500
```

```
    ELSE
        totalStorage = 2000
    ENDIF
    remainingStorage = totalStorage - usedStorage
    IF usedStorage > totalStorage THEN
        extraStorage = usedStorage - totalStorage
        extraCharges = extraStorage × 4
        remainingStorage = 0
    ELSE
        extraCharges = 0
    ENDIF
    usagePercentage =
    (usedStorage × 100)/totalStorage
    IF usagePercentage > 95 THEN
        PRINT "Storage Warning Generated."
    ENDIF
    PRINT usedStorage
    PRINT remainingStorage
    PRINT extraCharges
END FOR
STOP
```

C PROGRAM:

```
#include <stdio.h>
#define CUSTOMERS 50
int main()
{
    int i, planType;
    float totalStorage, usedStorage;
    float remainingStorage;
    float extraStorage, extraCharges;
```

```
float usagePercentage;
for(i = 1; i <= CUSTOMERS; i++)
{
   printf("\nCustomer %d\n", i);
   printf("Enter Plan Type ");
   printf("(1-Basic, 2-Standard, 3-Enterprise): ");
   scanf("%d",&planType);
   // Inactive account
   if(planType == -1)
   {
      printf("Inactive Account! Skipping...\n");
      continue;
   }
   // Server storage full
   if(planType == 999)
   {
      printf("SERVER STORAGE FULL!\n");
      break;
   }
   printf("Enter Used Storage (GB): ");
   scanf("%f",&usedStorage);
   // Assign plan storage
   if(planType == 1)
   {
      totalStorage = 100;
   }
   else if(planType == 2)
   {
      totalStorage = 500;
   }
   else
   {
```

```
    totalStorage = 2000;
}

remainingStorage =
totalStorage - usedStorage;

// Extra storage charges
if(usedStorage > totalStorage)
{
    extraStorage =
    usedStorage - totalStorage;

    extraCharges =
    extraStorage * 4;

    remainingStorage = 0;
}
else
{
    extraCharges = 0;
}

usagePercentage =
(usedStorage * 100) / totalStorage;

printf("Used Storage = %.2f GB\n",
        usedStorage);

printf("Remaining Storage = %.2f GB\n",
        remainingStorage);

printf("Extra Charges = Rs. %.2f\n",
```

```
            extraCharges);

        // Warning generation
        if(usagePercentage > 95)
        {
            printf("WARNING : Storage Usage "
                   "Exceeded 95%%!\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
Customer 1
Enter Plan Type (1-Basic, 2-Standard, 3-Enterprise): 1
Enter Used Storage (GB): 80
Used Storage = 80.00 GB
Remaining Storage = 20.00 GB
Extra Charges = Rs. 0.00

Customer 2
Enter Plan Type (1-Basic, 2-Standard, 3-Enterprise): 2
Enter Used Storage (GB): 490
Used Storage = 490.00 GB
Remaining Storage = 10.00 GB
Extra Charges = Rs. 0.00
WARNING : Storage Usage Exceeded 95%!

Customer 3
Enter Plan Type (1-Basic, 2-Standard, 3-Enterprise): 3
Enter Used Storage (GB): 2200
Used Storage = 2200.00 GB
Remaining Storage = 0.00 GB
Extra Charges = Rs. 800.00
WARNING : Storage Usage Exceeded 95%!
```

8. Smart Railway Reservation System
A railway coach contains 72 seats.
Reservation Rules:
Waiting List up to 20 → Allowed
Above 20 → Booking Closed
Senior citizens receive 40% fare concession, and students receive 20% concession.
(a)
Develop a C program using arithmetic operators to calculate the final ticket fare. Use decision-making statements to determine booking status and concession.
(b)
Use looping constructs to process passenger reservations. Use continue for invalid bookings and break when all seats are filled.

PSEUDOCODE

```
START
SET TOTAL_SEATS = 72
SET bookedSeats = 0
FOR each passenger
   INPUT baseFare
   IF baseFare = -1 THEN
      PRINT "Invalid Booking."
      CONTINUE
   ENDIF
   IF bookedSeats = TOTAL_SEATS THEN
      PRINT "All Seats Filled."
      BREAK
   ENDIF
   INPUT passengerType
   INPUT waitingListNumber
   IF waitingListNumber > 20 THEN
      PRINT "Booking Closed."
   ELSE
      IF passengerType = 1 THEN
         finalFare = baseFare - (40% of baseFare)
      ELSE IF passengerType = 2 THEN
         finalFare = baseFare - (20% of baseFare)
      ELSE
```

```
            finalFare = baseFare
        ENDIF
        IF waitingListNumber = 0 THEN
            bookedSeats = bookedSeats + 1
            PRINT "Confirmed Ticket"
        ELSE
            PRINT "Waiting List Allowed"
        ENDIF
        PRINT finalFare
    ENDIF
END FOR
STOP
```

C PROGRAM:

```
#include <stdio.h>

#define TOTAL_SEATS 72
#define PASSENGERS 100

int main()
{
    int i;
    int passengerType, waitingList;
    int bookedSeats = 0;
    float baseFare, finalFare;

    for(i = 1; i <= PASSENGERS; i++)
    {
        printf("\nPassenger %d\n", i);

        printf("Enter Base Fare (Rs.): ");
        scanf("%f",&baseFare);
```

```
// Invalid booking
if(baseFare == -1)
{
   printf("Invalid Booking! Skipping...\n");
   continue;
}
// All seats filled
if(bookedSeats == TOTAL_SEATS)
{
   printf("ALL SEATS ARE FILLED!\n");
   break;
}
printf("Passenger Type ");
printf("(1-Senior Citizen, 2-Student, 3-Regular): ");
scanf("%d",&passengerType);
printf("Enter Waiting List Number ");
printf("(0 for Confirmed Ticket): ");
scanf("%d",&waitingList);
// Booking closed
if(waitingList > 20)
{
   printf("Booking Closed!\n");
   continue;
}
// Fare concession
if(passengerType == 1)
{
   finalFare = baseFare - (0.40 * baseFare);
}
else if(passengerType == 2)
{
   finalFare = baseFare - (0.20 * baseFare);
```

```
        }
        else
        {
            finalFare = baseFare;
        }
        // Reservation status
        if(waitingList == 0)
        {
            bookedSeats++;
            printf("Status : Confirmed Ticket\n");
        }
        else
        {
            printf("Status : Waiting List Allowed\n");
        }
       printf("Final Ticket Fare = Rs. %.2f\n",
                finalFare);

        printf("Seats Filled = %d\n", bookedSeats);
    }
    return 0;
}
```

OUTPUT:

```
Passenger 1
Enter Base Fare (Rs.): 1000
Passenger Type (1-Senior Citizen, 2-Student, 3-Regular): 1
Enter Waiting List Number (0 for Confirmed Ticket): 0
Status : Confirmed Ticket
Final Ticket Fare = Rs. 600.00
Seats Filled = 1

Passenger 2
Enter Base Fare (Rs.): 800
Passenger Type (1-Senior Citizen, 2-Student, 3-Regular): 2
Enter Waiting List Number (0 for Confirmed Ticket): 0
Status : Confirmed Ticket
Final Ticket Fare = Rs. 640.00
Seats Filled = 2
```

9. Textile Manufacturing Quality Inspection (2026)
A textile factory produces 8,000 fabric rolls every month.
Inspection Rules:
Defects ≤2% → Premium Quality
2–5% → Standard Quality
Above 5% → Rework Required
(a)
Develop a C program using arithmetic operators to calculate the defect percentage and accepted rolls. Use decision-making statements to classify fabric quality.
(b)
Use looping constructs to process inspection details for 12 months. Use continue for incomplete inspections and break if production is suspended.

PSEUDOCODE:

```
START
SET TOTAL_ROLLS = 8000
FOR month = 1 TO 12
   INPUT defectiveRolls
   IF defectiveRolls = -1 THEN
      PRINT "Incomplete Inspection."
      CONTINUE
   ENDIF
   IF defectiveRolls = 999 THEN
      PRINT "Production Suspended!"
      BREAK
   ENDIF
   defectPercentage =
   (defectiveRolls × 100)/8000
   acceptedRolls =
   8000 - defectiveRolls

   IF defectPercentage <= 2 THEN
      PRINT "Premium Quality"
   ELSE IF defectPercentage <= 5 THEN
      PRINT "Standard Quality"
   ELSE
      PRINT "Rework Required"
   ENDIF
```

```
   PRINT defectPercentage
   PRINT acceptedRolls
END FOR
STOP
```

C PROGRAM:

```
#include <stdio.h>
#define TOTAL_ROLLS 8000
#define MONTHS 12
int main()
{
   int i, defectiveRolls, acceptedRolls;
   float defectPercentage;
   for(i = 1; i <= MONTHS; i++)
   {
      printf("\nMonth %d\n", i);
      printf("Enter Number of Defective Rolls : ");
      scanf("%d", &defectiveRolls);
      // Incomplete inspection
      if(defectiveRolls == -1)
      {
         printf("Incomplete Inspection! Skipping...\n");
         continue;
      }
      // Production suspended
      if(defectiveRolls == 999)
      {
         printf("PRODUCTION SUSPENDED!\n");
         break;
      }
      defectPercentage =
      ((float)defectiveRolls * 100)
      / TOTAL_ROLLS;
```

```
        acceptedRolls =
        TOTAL_ROLLS - defectiveRolls;
        printf("Defect Percentage = %.2f%%\n",
              defectPercentage);
        printf("Accepted Rolls = %d\n",
              acceptedRolls);
        // Quality Classification
        if(defectPercentage <= 2)
        {
           printf("Quality : Premium Quality\n");
        }
        else if(defectPercentage <= 5)
        {
           printf("Quality : Standard Quality\n");
        }
        else
        {
           printf("Quality : Rework Required\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
Month 1
Enter Number of Defective Rolls : 100
Defect Percentage = 1.25%
Accepted Rolls = 7900
Quality : Premium Quality

Month 2
Enter Number of Defective Rolls : 200
Defect Percentage = 2.50%
Accepted Rolls = 7800
Quality : Standard Quality
```

10. AI-Based Traffic Signal Monitoring System
A smart city monitors traffic density at 50 intersections every hour.
Traffic Classification:
≤100 vehicles/hour → Low Traffic
101–300 → Moderate Traffic
301–500 → Heavy Traffic
Above 500 → Congestion Alert
If congestion continues for 3 consecutive hours, an emergency traffic diversion should be initiated.
(a)
Develop a C program using arithmetic operators to calculate the average traffic density. Use decision-making statements to classify each intersection and determine whether a congestion alert should be generated.
(b)
Use looping constructs to process traffic data from all 50 intersections. Use continue to skip faulty sensors and break when emergency traffic control is activated.

PSEUDOCODE:

```
START
SET totalTraffic = 0
FOR intersection = 1 TO 50
   INPUT trafficDensity
   IF trafficDensity = -1 THEN
      PRINT "Faulty Sensor."
      CONTINUE
   ENDIF
   IF trafficDensity = 999 THEN
      PRINT "Emergency Traffic Control Activated!"
      BREAK
   ENDIF
   totalTraffic = totalTraffic + trafficDensity
   IF trafficDensity <= 100 THEN
      PRINT "Low Traffic"

   ELSE IF trafficDensity <= 300 THEN
      PRINT "Moderate Traffic"
   ELSE IF trafficDensity <= 500 THEN
      PRINT "Heavy Traffic"
   ELSE
      PRINT "Congestion Alert"
```

```
    ENDIF
    IF trafficDensity > 500 THEN
        PRINT "Congestion Alert Generated."
        PRINT "If congestion continues for 3 hours,"
        PRINT "Emergency Traffic Diversion Required."
    ENDIF
END FOR
averageTraffic = totalTraffic / 50
PRINT averageTraffic
STOP
```

C PROGRAM:

```
#include <stdio.h>
#define INTERSECTIONS 50
int main()
{
    int i, trafficDensity;
    float totalTraffic = 0;
    float averageTraffic;

    for(i = 1; i <= INTERSECTIONS; i++)
    {
        printf("\nIntersection %d\n", i);
        78printf("Enter Traffic Density (vehicles/hour): ");
        scanf("%d", &trafficDensity);
        // Faulty sensor
        if(trafficDensity == -1)
        {
            printf("Faulty Sensor! Skipping...\n");
            continue;
        }
        // Emergency traffic control
```

```
    if(trafficDensity == 999)
    {
        printf("EMERGENCY TRAFFIC CONTROL ACTIVATED!\n");
        break;
    }
    totalTraffic += trafficDensity;
    // Traffic Classification
    if(trafficDensity <= 100)
    {
        printf("Traffic Status : Low Traffic\n");
    }
    else if(trafficDensity <= 300)
    {
        printf("Traffic Status : Moderate Traffic\n");}
    else if(trafficDensity <= 500)
    {
        printf("Traffic Status : Heavy Traffic\n");
    }
    else
    {
        printf("Traffic Status : Congestion Alert\n");
    }
    // Congestion Warning
    if(trafficDensity > 500)
    {
        printf("Congestion Alert Generated!\n");
        printf("Emergency Traffic Diversion "
               "Required if it continues "
               "for 3 consecutive hours.\n");
    }
}
averageTraffic =
```

```
    totalTraffic / INTERSECTIONS;
    printf("\n========== Traffic Report ==========\n");
    printf("Average Traffic Density = %.2f vehicles/hour\n",
            averageTraffic);
    return 0;
}
```

OUTPUT:

```
Intersection 1
Enter Traffic Density (vehicles/hour): 85
Traffic Status : Low Traffic

Intersection 2
Enter Traffic Density (vehicles/hour): 350
Traffic Status : Heavy Traffic

Intersection 3
Enter Traffic Density (vehicles/hour): 450
Traffic Status : Heavy Traffic
```

11. Smart Data Center Cooling Management System

A cloud computing company operates 20 data centers during 2026–2027. Each data center should maintain a temperature between 18°C and 27°C. If the temperature exceeds 35°C, the cooling system enters emergency mode. If the temperature falls below 15°C, an energy-saving alert is generated.

(a)

Develop a C program using arithmetic operators to calculate the average temperature recorded in each data center. Use decision-making statements (if, else if, else) to classify the cooling status as Normal, Warning, Emergency, or Energy Saving.

(b)

Use looping constructs (for/while) to process temperature readings from all 20 data centers. Use continue to skip faulty sensor readings and break if emergency shutdown is initiated.

PSEUDO CODE:

```
START
SET totalTemperature = 0
FOR dataCenter = 1 TO 20
    INPUT temperature
    IF temperature = -1 THEN
        PRINT "Faulty Sensor Reading."
        CONTINUE
    ENDIF
    IF temperature = 999 THEN
        PRINT "Emergency Shutdown Initiated!"
        BREAK
    ENDIF
    totalTemperature = totalTemperature + temperature
    IF temperature < 15 THEN
        PRINT "Energy Saving Alert"
    ELSE IF temperature >= 18 AND temperature <= 27 THEN
        PRINT "Normal"
    ELSE IF temperature > 35 THEN
        PRINT "Emergency Mode"
    ELSE
        PRINT "Warning"
    ENDIF
END FOR
averageTemperature = totalTemperature / 20
```

PRINT averageTemperature

STOP

C PROGRAM:

```
#include <stdio.h>
#define DATACENTERS 20
int main()
{
   int i;
   float temperature;
   float totalTemperature = 0;
   float averageTemperature;
   for(i = 1; i <= DATACENTERS; i++)
   {
      printf("\nData Center %d\n", i);
      printf("Enter Temperature (°C): ");
      scanf("%f", &temperature);
      // Faulty sensor reading
      if(temperature == -1)
      {
         printf("Faulty Sensor Reading! Skipping...\n");
         continue;
      }
      // Emergency shutdown
      if(temperature == 999)
      {
         printf("EMERGENCY SHUTDOWN INITIATED!\n");
         break;
      }

      totalTemperature += temperature;
```

```
        // Cooling status classification
        if(temperature < 15)
        {
            printf("Cooling Status : Energy Saving Alert\n");
        }
        else if(temperature >= 18 &&
                temperature <= 27)
        {
            printf("Cooling Status : Normal\n");
        }
        else if(temperature > 35)
        {
           printf("Cooling Status : Emergency Mode\n");
        }
        else
        {
            printf("Cooling Status : Warning\n");
        }
    }
    averageTemperature =
    totalTemperature / DATACENTERS;

    printf("\n========== Cooling Report ==========\n");
    printf("Average Temperature = %.2f °C\n",
            averageTemperature);

    return 0;
}
```

OUTPUT:

```
Data Center 1
Enter Temperature (°C): 22
Cooling Status : Normal

Data Center 2
Enter Temperature (°C): 30
Cooling Status : Warning

Data Center 3
Enter Temperature (°C): 38
Cooling Status : Emergency Mode

Data Center 4
Enter Temperature (°C): 12
Cooling Status : Energy Saving Alert
```

12. Steel Manufacturing Quality Control System

A steel manufacturing plant produces 15,000 steel rods per day. During inspection, rods are tested for strength.

Quality Standards:

≥95% Strength → Grade A

85–94% → Grade B

70–84% → Grade C

Below 70% → Rejected

(a)

Develop a C program using arithmetic operators to calculate the percentage of accepted and rejected rods. Use decision-making statements to classify the quality grade.

(b)

Use looping constructs to process inspection data for 30 production days. Use continue to ignore damaged test reports and break if the rejection rate exceeds 20%.

PSEUDOCODE:

```
START
SET TOTAL_RODS = 15000
FOR day = 1 TO 30
   INPUT strengthPercentage
   IF strengthPercentage = -1 THEN
      PRINT "Damaged Test Report."
      CONTINUE
   ENDIF
   IF strengthPercentage < 70 THEN
      rejectedRods = TOTAL_RODS
      rejectionPercentage = 100
      PRINT "Rejected"
      IF rejectionPercentage > 20 THEN
         PRINT "Inspection Stopped."
         BREAK
      ENDIF
   ELSE
      acceptedRods = TOTAL_RODS
      rejectedRods = 0
      IF strengthPercentage >= 95 THEN
         PRINT "Grade A"
      ELSE IF strengthPercentage >= 85 THEN
```

```
        PRINT "Grade B"
      ELSE
        PRINT "Grade C"
      ENDIF
   ENDIF
   acceptedPercentage =
   (acceptedRods × 100)/TOTAL_RODS
   rejectionPercentage =
   (rejectedRods × 100)/TOTAL_RODS
   PRINT acceptedPercentage
   PRINT rejectionPercentage
END FOR
STOP
```

C PROGRAM:

```
#include <stdio.h>

#define TOTAL_RODS 15000
#define DAYS 30

int main()
{
   int i;
   float strengthPercentage;
   int acceptedRods, rejectedRods;
   float acceptedPercentage, rejectionPercentage;

   for(i = 1; i <= DAYS; i++)
   {
      printf("\nProduction Day %d\n", i);

      printf("Enter Strength Percentage : ");
```

```
scanf("%f", &strengthPercentage);

// Damaged test report
if(strengthPercentage == -1)
{
   printf("Damaged Test Report! Skipping...\n");
   continue;
}

// Quality Classification
if(strengthPercentage >= 95)
{
   acceptedRods = TOTAL_RODS;
   rejectedRods = 0;

   printf("Quality Grade : Grade A\n");
}
else if(strengthPercentage >= 85)
{
   acceptedRods = TOTAL_RODS;
   rejectedRods = 0;

   printf("Quality Grade : Grade B\n");
}
else if(strengthPercentage >= 70)
{
   acceptedRods = TOTAL_RODS;
   rejectedRods = 0;

   printf("Quality Grade : Grade C\n");
}
else
```

```
        {
            acceptedRods = 0;
            rejectedRods = TOTAL_RODS;

            printf("Quality Grade : REJECTED\n");
        }

        acceptedPercentage =
        ((float)acceptedRods * 100)
        / TOTAL_RODS;

        rejectionPercentage =
        ((float)rejectedRods * 100)
        / TOTAL_RODS;

        printf("Accepted Percentage = %.2f%%\n",
                acceptedPercentage);

        printf("Rejected Percentage = %.2f%%\n",
                rejectionPercentage);

        // Break condition
        if(rejectionPercentage > 20)
        {
            printf("REJECTION RATE EXCEEDED 20%%!\n");
            printf("Inspection Process Stopped.\n");
            break;
        }
    }

    return 0;
}
```

OUTPUT:

```
Production Day 1
Enter Strength Percentage : 98
Quality Grade : Grade A
Accepted Percentage = 100.00%
Rejected Percentage = 0.00%

Production Day 2
Enter Strength Percentage : 90
Quality Grade : Grade B
Accepted Percentage = 100.00%
Rejected Percentage = 0.00%

Production Day 3
Enter Strength Percentage : 75
Quality Grade : Grade C
Accepted Percentage = 100.00%
Rejected Percentage = 0.00%
```

13. Smart Retail Sales Performance System
A retail chain has 50 branches across India. Each branch has a monthly sales target of ₹50,00,000.
Performance Criteria:
≥110% of target → Outstanding
90–109% → Achieved Target
70–89% → Improvement Required
Below 70% → Critical
(a)
Develop a C program using arithmetic operators to calculate sales achievement percentage and incentive amount. Use decision-making statements to classify branch performance.
(b)
Use looping constructs to process monthly sales data for all branches. Use continue to skip newly opened branches and break when the financial audit begins.

PSEUDO CODE:

```
START

SET TARGET = 5000000

FOR branch = 1 TO 50

   INPUT monthlySales

   IF monthlySales = -1 THEN
      PRINT "Newly Opened Branch."
      CONTINUE
   ENDIF

   IF monthlySales = 999 THEN
      PRINT "Financial Audit Began."
      BREAK
   ENDIF

   salesPercentage =
   (monthlySales × 100)/TARGET

   incentive = 0
```

```
    IF salesPercentage >= 110 THEN

        status = Outstanding
        incentive = TARGET × 10 /100

    ELSE IF salesPercentage >= 90 THEN

        status = Achieved Target

    ELSE IF salesPercentage >=70 THEN

        status = Improvement Required

    ELSE

        status = Critical

    ENDIF

    PRINT salesPercentage
    PRINT incentive
    PRINT status

END FOR

STOP
```

C PROGRAM:

```
#include <stdio.h>

#define BRANCHES 50
#define TARGET 5000000
```

```
int main()
{
    int i;
    float monthlySales;
    float salesPercentage;
    float incentive = 0;

    for(i=1;i<=BRANCHES;i++)
    {
        printf("\nBranch %d\n",i);

        printf("Enter Monthly Sales (Rs.): ");
        scanf("%f",&monthlySales);

        // Newly opened branch
        if(monthlySales == -1)
        {
            printf("Newly Opened Branch! Skipping...\n");
            continue;
        }

        // Financial audit
        if(monthlySales == 999)
        {
            printf("FINANCIAL AUDIT HAS BEGUN!\n");
            break;
        }

        salesPercentage =
        (monthlySales * 100) / TARGET;
```

```
        incentive = 0;

        if(salesPercentage >= 110)
        {
            incentive = TARGET * 0.10;
            printf("Performance : Outstanding\n");
        }
        else if(salesPercentage >= 90)
        {
            printf("Performance : Achieved Target\n");
        }
        else if(salesPercentage >= 70)
        {
            printf("Performance : Improvement Required\n");
        }
        else
        {
            printf("Performance : Critical\n");
        }

        printf("Sales Achievement = %.2f%%\n",
                salesPercentage);

        printf("Incentive Amount = Rs. %.2f\n",
                incentive);
    }

    return 0;
}
```

OUTPUT:

```
Branch 1
Enter Monthly Sales (Rs.): 600000
Performance : Critical
Sales Achievement = 12.00%
Incentive Amount = Rs. 0.00

Branch 2
Enter Monthly Sales (Rs.): 400000
Performance : Critical
Sales Achievement = 8.00%
Incentive Amount = Rs. 0.00
```

14. Industrial Boiler Safety Monitoring System
A thermal power plant monitors 10 industrial boilers every hour.
Safety Conditions:
Pressure: 120–180 PSI
Temperature: 250°C–450°C
If pressure exceeds 200 PSI or temperature exceeds 500°C, the boiler should be shut down immediately.
(a)
Develop a C program using arithmetic operators to calculate average pressure and temperature. Use decision-making statements to determine the boiler status.
(b)
Use looping constructs to monitor all boilers continuously. Use continue for boilers under maintenance and break during emergency shutdown.

PSEUDO CODE:

```
START

SET totalPressure = 0
SET totalTemperature = 0

FOR boiler = 1 TO 10

   INPUT pressure

   IF pressure = -1 THEN
      PRINT "Boiler Under Maintenance."
      CONTINUE
   ENDIF

   IF pressure = 999 THEN
      PRINT "Emergency Shutdown Initiated!"
      BREAK
   ENDIF

   INPUT temperature

   totalPressure = totalPressure + pressure
```

```
    totalTemperature = totalTemperature + temperature

    IF pressure > 200 OR temperature > 500 THEN

        PRINT "EMERGENCY SHUTDOWN"

    ELSE IF pressure >= 120 AND pressure <=180
    AND temperature >=250 AND temperature <=450 THEN

        PRINT "NORMAL"

    ELSE

        PRINT "WARNING"

    ENDIF

END FOR

averagePressure = totalPressure / 10
averageTemperature = totalTemperature / 10

PRINT averagePressure
PRINT averageTemperature

STOP
```

C PROGRAM:

```
#include <stdio.h>

#define BOILERS 10
```

```
int main()
{
    int i;
    float pressure, temperature;
    float totalPressure = 0;
    float totalTemperature = 0;
    float averagePressure;
    float averageTemperature;

    for(i = 1; i <= BOILERS; i++)
    {
        printf("\nBoiler %d\n", i);

        printf("Enter Pressure (PSI): ");
        scanf("%f", &pressure);

        // Boiler under maintenance
        if(pressure == -1)
        {
            printf("Boiler Under Maintenance! Skipping...\n");
            continue;
        }

        // Emergency shutdown initiated
        if(pressure == 999)
        {
            printf("EMERGENCY SHUTDOWN INITIATED!\n");
            break;
        }

        printf("Enter Temperature (°C): ");
        scanf("%f", &temperature);
```

```
    totalPressure += pressure;
    totalTemperature += temperature;

    // Boiler status
    if(pressure > 200 || temperature > 500)
    {
        printf("Status : EMERGENCY SHUTDOWN\n");
    }
    else if((pressure >= 120 &&
             pressure <= 180) &&
            (temperature >= 250 &&
             temperature <= 450))
    {
        printf("Status : NORMAL\n");
    }
    else
    {
        printf("Status : WARNING\n");
    }
}

averagePressure =
totalPressure / BOILERS;

averageTemperature =
totalTemperature / BOILERS;

printf("\n========== SAFETY REPORT ==========\n");

printf("Average Pressure = %.2f PSI\n",
       averagePressure);
```

```
    printf("Average Temperature = %.2f °C\n",
            averageTemperature);

    return 0;
}
```

OUTPUT:

```
Boiler 1
Enter Pressure (PSI): 120
Enter Temperature (°C): 350
Status : NORMAL

Boiler 2
Enter Pressure (PSI): 170
Enter Temperature (°C): 430
Status : NORMAL

Boiler 3
Enter Pressure (PSI): 195
Enter Temperature (°C): 470
Status : WARNING
```

15. Banking Fixed Deposit Interest Calculation System
A national bank offers fixed deposits with the following annual interest rates:
Less than 1 year → 5.5%
1–3 years → 6.5%
3–5 years → 7.25%
More than 5 years → 8%
The bank processes 500 customer deposits every month.
(a)
Develop a C program using arithmetic operators to calculate simple interest, maturity amount, and total returns. Use decision-making statements to determine the applicable interest rate.
(b)
Use looping constructs to process multiple customer deposits. Use continue for incomplete applications and break when the banking session ends.

PSEUDO CODE:

```
START

FOR customer = 1 TO 500

   INPUT principalAmount

   IF principalAmount = -1 THEN
      PRINT "Incomplete Application."
      CONTINUE
   ENDIF

   IF principalAmount = 999 THEN
      PRINT "Banking Session Ended."
      BREAK
   ENDIF

   INPUT time (in years)

   IF time < 1 THEN
      rate = 5.5

   ELSE IF time <= 3 THEN
      rate = 6.5
```

```
    ELSE IF time <= 5 THEN
        rate = 7.25

    ELSE
        rate = 8.0
    ENDIF

    simpleInterest =
    (principal × rate × time)/100

    maturityAmount =
    principal + simpleInterest

    PRINT rate
    PRINT simpleInterest
    PRINT maturityAmount

END FOR

STOP
```

C PROGRAM:

```
#include <stdio.h>

#define CUSTOMERS 500

int main()
{
    int i;
    float principal, time, rate;
    float simpleInterest;
```

```
float maturityAmount;

for(i = 1; i <= CUSTOMERS; i++)
{
   printf("\nCustomer %d\n", i);

   printf("Enter Principal Amount (Rs.): ");
   scanf("%f", &principal);

   // Incomplete application
   if(principal == -1)
   {
      printf("Incomplete Application! Skipping...\n");
      continue;
   }

   // Banking session ends
   if(principal == 999)
   {
      printf("BANKING SESSION HAS ENDED!\n");
      break;
   }

   printf("Enter Time Period (Years): ");
   scanf("%f", &time);

   // Interest rate determination
   if(time < 1)
   {
      rate = 5.5;
   }
   else if(time <= 3)
```

```
        {
            rate = 6.5;
        }
        else if(time <= 5)
        {
            rate = 7.25;
        }
        else
        {
            rate = 8.0;
        }

        // Calculations
        simpleInterest =
        (principal * rate * time) / 100;

        maturityAmount =
        principal + simpleInterest;

        printf("Applicable Interest Rate = %.2f%%\n",
                rate);

        printf("Simple Interest = Rs. %.2f\n",
                simpleInterest);

        printf("Maturity Amount = Rs. %.2f\n",
                maturityAmount);
    }

    return 0;
}
```

OUTPUT:

```
Customer 1
Enter Principal Amount (Rs.): 5000
Enter Time Period (Years): 3
Applicable Interest Rate = 6.50%
Simple Interest = Rs. 975.00
Maturity Amount = Rs. 5975.00

Customer 2
Enter Principal Amount (Rs.): 600000
Enter Time Period (Years): 5
Applicable Interest Rate = 7.25%
Simple Interest = Rs. 217500.00
Maturity Amount = Rs. 817500.00
```

16. Smart Construction Material Management System

A construction company manages cement deliveries for 100 building sites. Each site requires 500 cement bags per week.

Material Status:

≥500 bags → Adequate

300–499 bags → Low Stock

Below 300 bags → Urgent Supply Required

(a)

Develop a C program using arithmetic operators to calculate remaining cement stock after usage. Use decision-making statements to classify stock availability.

(b)

Use looping constructs to process all construction sites. Use continue to skip inactive sites and break if warehouse stock becomes empty.

PSEUDO CODE:

```
START

FOR site = 1 TO 100

   INPUT availableStock

   IF availableStock = -1 THEN
      PRINT "Inactive Construction Site."
      CONTINUE
   ENDIF

   IF availableStock = 999 THEN
      PRINT "Warehouse Stock Empty."
      BREAK
   ENDIF

   INPUT cementUsed

   remainingStock =
   availableStock - cementUsed

   IF remainingStock >= 500 THEN
      PRINT "Adequate"
```

```
    ELSE IF remainingStock >= 300 THEN
        PRINT "Low Stock"

    ELSE
        PRINT "Urgent Supply Required"

    ENDIF

    PRINT remainingStock

END FOR

STOP
```

C PROGRAM:

```
#include <stdio.h>

#define SITES 100

int main()
{
    int i;
    int availableStock, cementUsed;
    int remainingStock;

    for(i = 1; i <= SITES; i++)
    {
        printf("\nConstruction Site %d\n", i);

        printf("Enter Available Cement Bags : ");
        scanf("%d", &availableStock);
```

```
// Inactive site
if(availableStock == -1)
{
   printf("Inactive Construction Site! Skipping...\n");
   continue;
}

// Warehouse stock becomes empty
if(availableStock == 999)
{
   printf("WAREHOUSE STOCK HAS BECOME EMPTY!\n");
   break;
}

printf("Enter Cement Bags Used : ");
scanf("%d", &cementUsed);

// Arithmetic calculation
remainingStock =
availableStock - cementUsed;

// Material classification
if(remainingStock >= 500)
{
   printf("Material Status : Adequate\n");
}
else if(remainingStock >= 300)
{
   printf("Material Status : Low Stock\n");
}
else
```

```
        {
            printf("Material Status : Urgent Supply Required\n");
        }

        printf("Remaining Stock = %d Bags\n",
                remainingStock);
    }

    return 0;
}
```

OUTPUT:

```
Construction Site 1
Enter Available Cement Bags : 100
Enter Cement Bags Used : 20
Material Status : Urgent Supply Required
Remaining Stock = 80 Bags

Construction Site 2
Enter Available Cement Bags : 1000
Enter Cement Bags Used : 400
Material Status : Adequate
Remaining Stock = 600 Bags
```

17. AI-Based Customer Support Performance System
A BPO company evaluates 250 customer support executives every month.
Performance Indicators:
Customer Satisfaction ≥95%
Resolution Time ≤5 minutes
Attendance ≥98%
Overall Rating:
Excellent
Good
Average
Poor
(a)
Develop a C program using arithmetic operators to calculate average performance score. Use decision-making statements to determine the employee rating.
(b)
Use looping constructs to process all executives. Use continue to ignore employees on leave and break when HR completes the monthly review.

PSEUDO CODE:

```
START

FOR employee = 1 TO 250

   INPUT customerSatisfaction

   IF customerSatisfaction = -1 THEN
      PRINT "Employee is on Leave."
      CONTINUE
   ENDIF

   IF customerSatisfaction = 999 THEN
      PRINT "Monthly Review Completed."
      BREAK
   ENDIF

   INPUT resolutionTime
   INPUT attendance

   IF resolutionTime <= 5 THEN
```

```
        resolutionScore = 100
    ELSE
        resolutionScore = 80
    ENDIF

    averageScore =
    (customerSatisfaction +
     attendance +
     resolutionScore) / 3

    IF averageScore >= 95 THEN
        PRINT "Excellent"

    ELSE IF averageScore >= 85 THEN
        PRINT "Good"

    ELSE IF averageScore >= 70 THEN
        PRINT "Average"

    ELSE
        PRINT "Poor"

    ENDIF

    PRINT averageScore

END FOR

STOP
```

C PROGRAM:

```
#include <stdio.h>
```

```
#define EMPLOYEES 250

int main()
{
   int i;
   float customerSatisfaction;
   float attendance;
   float resolutionTime;
   float resolutionScore;
   float averageScore;

   for(i = 1; i <= EMPLOYEES; i++)
   {
      printf("\nEmployee %d\n", i);

      printf("Enter Customer Satisfaction (%%): ");
      scanf("%f", &customerSatisfaction);

      // Employee on leave
      if(customerSatisfaction == -1)
      {
         printf("Employee is on Leave! Skipping...\n");
         continue;
      }

      // Monthly review completed
      if(customerSatisfaction == 999)
      {
         printf("HR HAS COMPLETED THE MONTHLY REVIEW!\n");
         break;
      }
```

```
printf("Enter Resolution Time (minutes): ");
scanf("%f", &resolutionTime);

printf("Enter Attendance (%%): ");
scanf("%f", &attendance);

// Resolution score calculation
if(resolutionTime <= 5)
{
   resolutionScore = 100;
}
else
{
   resolutionScore = 80;
}

// Average performance score
averageScore =
(customerSatisfaction +
 attendance +
 resolutionScore) / 3;

// Rating determination
if(averageScore >= 95)
{
   printf("Overall Rating : Excellent\n");
}
else if(averageScore >= 85)
{
   printf("Overall Rating : Good\n");
}
```

```
        else if(averageScore >= 70)
        {
            printf("Overall Rating : Average\n");
        }
        else
        {
            printf("Overall Rating : Poor\n");
        }

        printf("Average Performance Score = %.2f\n",
                averageScore);
    }

    return 0;
}
```

OUTPUT:

```
Employee 1
Enter Customer Satisfaction (%): 98
Enter Resolution Time (minutes): 4
Enter Attendance (%): 99
Overall Rating : Excellent
Average Performance Score = 99.00

Employee 2
Enter Customer Satisfaction (%): 90
Enter Resolution Time (minutes): 6
Enter Attendance (%): 95
Overall Rating : Good
Average Performance Score = 88.33
```

18. Smart Cold Storage Monitoring System
A food processing company stores fruits and vegetables in 30 cold storage units.
Required Temperature:
Fruits: 2°C–8°C
Vegetables: 4°C–10°C
If the temperature exceeds 12°C, food spoilage warning should be generated.
(a)
Develop a C program using arithmetic operators to calculate average storage temperature. Use decision-making statements to classify each storage unit as Safe, Warning, or Critical.
(b)
Use looping constructs to process temperature readings from all storage units. Use continue to ignore faulty sensors and break if refrigeration fails.

PSEUDO CODE:

```
START
   Initialize total_temperature = 0
   Initialize valid_units_count = 0

   FOR unit = 1 TO 30 DO
      PRINT "Enter storage type (1 for Fruit, 2 for Vegetable) and Temperature:"
      READ unit_type, temperature

      IF temperature == -999 THEN
         PRINT "Faulty sensor detected. Skipping unit."
         CONTINUE
      ENDIF

      IF temperature > 40 THEN
         PRINT "Refrigeration failure detected! Stopping process."
         BREAK
      ENDIF

      Accumulate total_temperature = total_temperature + temperature
      Increment valid_units_count by 1

      IF unit_type == 1 THEN  // Fruits
         IF temperature >= 2 AND temperature <= 8 THEN
```

```
            PRINT "Status: Safe"
        ELSEIF temperature > 12 THEN
            PRINT "Status: Critical (Spoilage Warning)"
        ELSE
            PRINT "Status: Warning"
        ENDIF
    ELSEIF unit_type == 2 THEN  // Vegetables
        IF temperature >= 4 AND temperature <= 10 THEN
            PRINT "Status: Safe"
        ELSEIF temperature > 12 THEN
            PRINT "Status: Critical (Spoilage Warning)"
        ELSE
            PRINT "Status: Warning"
        ENDIF
    ENDIF
  ENDFOR

  IF valid_units_count > 0 THEN
    Calculate average_temperature = total_temperature / valid_units_count
    PRINT "Average Storage Temperature:", average_temperature
  ELSE
    PRINT "No valid data to calculate average."
  ENDIF
END
```

C PROGRAM:

```
#include <stdio.h>

int main() {
  int unit_type;
  float temperature;
  float total_temperature = 0;
```

```
    int valid_units_count = 0;

    printf("--- Smart Cold Storage Monitoring System ---\n");
    printf("Enter storage type: 1 for Fruits, 2 for Vegetables\n");
    printf("Enter temperature '-999' to simulate a faulty sensor.\n");
    printf("Enter temperature above '40' to simulate complete refrigeration failure.\n\n");

    for (int i = 1; i <= 30; i++) {
        printf("Storage Unit %d - Enter Type & Temperature: ", i);
        scanf("%d %f", &unit_type, &temperature);

        // (b) Use continue to ignore faulty sensors
        if (temperature == -999) {
            printf("-> Unit %d: Faulty sensor ignored.\n\n", i);
            continue;
        }

        // (b) Use break if refrigeration fails (extreme temperature threshold)
        if (temperature > 40.0) {
            printf("-> CRITICAL: Refrigeration failure detected at Unit %d! Terminating system
monitoring.\n\n", i);
            break;
        }

        // (a) Arithmetic accumulation for average
        total_temperature += temperature;
        valid_units_count++;

        // (a) Decision-making statements to classify units
        if (unit_type == 1) { // Fruits
            if (temperature >= 2.0 && temperature <= 8.0) {
                printf("-> Status: Safe\n\n");
```

```
            } else if (temperature > 12.0) {
                printf("-> Status: Critical (Food Spoilage Warning!)\n\n");
            } else {
                printf("-> Status: Warning\n\n");
            }
        }
        else if (unit_type == 2) { // Vegetables
            if (temperature >= 4.0 && temperature <= 10.0) {
                printf("-> Status: Safe\n\n");
            } else if (temperature > 12.0) {
                printf("-> Status: Critical (Food Spoilage Warning!)\n\n");
            } else {
                printf("-> Status: Warning\n\n");
            }
        }
        else {
            printf("-> Invalid storage type entered.\n\n");
        }
    }
    // (a) Calculate and display average storage temperature
    if (valid_units_count > 0) {
        float average_temp = total_temperature / valid_units_count;
        printf("----------------------------------------\n");
        printf("Total Valid Units Processed: %d\n", valid_units_count);
        printf("Average Storage Temperature: %.2f°C\n", average_temp);
        printf("----------------------------------------\n");
    } else {
        printf("No valid temperature readings recorded.\n");
    }

    return 0;
}
```

OUTPUT:

```
=== Smart Cold Storage Monitoring System ===
Enter storage unit details:
Type: 1 for Fruits (2°C - 8°C), 2 for Vegetables (4°C - 10°C)

Unit 1 - Enter type (1/2) and temperature (°C): 5.0
  -> Status: WARNING (Temperature out of safe range)
Unit 2 - Enter type (1/2) and temperature (°C): -10.9
  -> Status: WARNING (Temperature out of safe range)
Unit 3 - Enter type (1/2) and temperature (°C): 14.0
  -> Status: WARNING (Temperature out of safe range)
Unit 4 - Enter type (1/2) and temperature (°C): 35.0
  -> Status: WARNING (Temperature out of safe range)
```

19. Smart Waste Recycling Plant System
A recycling plant processes 5,000 kg of waste every day.
Waste Categories:
Plastic
Paper
Metal
Electronic Waste
Efficiency Levels:
≥90% Recycled → Excellent
75–89% → Good
50–74% → Average
Below 50% → Poor
(a) Develop a C program using arithmetic operators to calculate recycled waste percentage and remaining waste. Use decision-making statements to determine plant efficiency.
(b) Use looping constructs to process waste data for 30 days. Use continue to skip maintenance days and break if the processing machine fails.

PSEUDO CODE:

```
START
   CONSTANT TOTAL_DAILY_WASTE = 5000.0
   Initialize total_recycled_month = 0.0, active_days = 0

   FOR day = 1 TO 30 DO
      PRINT "Day day: Enter total recycled waste in kg (or -1 for Maintenance, -2 for Machine Failure):"
      READ recycled_kg

      // Skip maintenance days using continue
      IF recycled_kg == -1 THEN
         PRINT "Day day: Scheduled Maintenance Day. Skipping..."
         CONTINUE
      END IF

      // Stop processing on machine failure using break
      IF recycled_kg == -2 THEN
         PRINT "Day day: CRITICAL MACHINE FAILURE! Stopping operations."
         BREAK
      END IF
```

```
        // Calculate remaining waste and efficiency percentage
        remaining_kg = TOTAL_DAILY_WASTE - recycled_kg
        efficiency_pct = (recycled_kg / TOTAL_DAILY_WASTE) * 100

        // Update totals
        total_recycled_month = total_recycled_month + recycled_kg
        active_days = active_days + 1

        // Determine Efficiency Level
        IF efficiency_pct >= 90.0 THEN
            status = "Excellent"
        ELSE IF efficiency_pct >= 75.0 AND efficiency_pct <= 89.99 THEN
            status = "Good"
        ELSE IF efficiency_pct >= 50.0 AND efficiency_pct <= 74.99 THEN
            status = "Average"
        ELSE
            status = "Poor"
        END IF

        PRINT "Recycled: recycled_kg kg (efficiency_pct%)"
        PRINT "Remaining: remaining_kg kg"
        PRINT "Efficiency Status: status"
    END FOR

    // Display summary
    IF active_days > 0 THEN
        PRINT "Total Days Processed: active_days"
        PRINT "Total Recycled Waste: total_recycled_month kg"
    END IF
END
```

C PROGRAM:

```
#include <stdio.h>

int main() {
    const float TOTAL_DAILY_WASTE = 5000.0; // 5,000 kg per day
    float recycled_kg;
    float remaining_kg;
    float efficiency_pct;
    float total_recycled_month = 0.0;
    int active_days = 0;

    printf("=== Smart Waste Recycling Plant System ===\n");
    printf("Daily Target Waste Capacity: %.0f kg\n", TOTAL_DAILY_WASTE);
    printf("Special Codes: Enter '-1' for Maintenance Day | '-2' for Machine Failure\n\n");

    for (int day = 1; day <= 30; day++) {
        printf("--- Day %d ---\n", day);
        printf("Enter total recycled waste (kg): ");
        if (scanf("%f", &recycled_kg) != 1) {
            printf("Invalid input format. Exiting...\n");
            break;
        }

        // (b) Use 'continue' to skip maintenance days
        if (recycled_kg == -1.0) {
            printf("-> Status: Plant Maintenance Day. Skipping record...\n\n");
            continue;
        }

        // (b) Use 'break' if the processing machine fails
        if (recycled_kg == -2.0) {
```

```
            printf("-> ALERT: Processing Machine Failure detected on Day %d! Halting
operations.\n\n", day);
            break;
        }

        // (a) Calculate remaining waste and recycled percentage
        remaining_kg = TOTAL_DAILY_WASTE - recycled_kg;
        efficiency_pct = (recycled_kg / TOTAL_DAILY_WASTE) * 100.0;

        total_recycled_month += recycled_kg;
        active_days++;

        // Display daily waste metrics
        printf("  Recycled Waste  : %.2f kg (%.2f%%)\n", recycled_kg, efficiency_pct);
        printf("  Remaining Waste : %.2f kg\n", remaining_kg);

        // (a) Decision-making statements to determine plant efficiency
        if (efficiency_pct >= 90.0) {
            printf("  Efficiency Level: EXCELLENT\n\n");
        }
        else if (efficiency_pct >= 75.0) {
            printf("  Efficiency Level: GOOD\n\n");
        }
        else if (efficiency_pct >= 50.0) {
            printf("  Efficiency Level: AVERAGE\n\n");
        }
        else {
            printf("  Efficiency Level: POOR\n\n");
        }
    }

    // Monthly Operational Summary
```

```
    printf("==========================================\n");
    printf("            MONTHLY SUMMARY               \n");
    printf("==========================================\n");
    if (active_days > 0) {
        printf("Active Days Processed   : %d day(s)\n", active_days);
        printf("Total Waste Recycled    : %.2f kg\n", total_recycled_month);
        printf("Average Daily Recycled  : %.2f kg\n", total_recycled_month / active_days);
    } else {
        printf("No active processing days recorded.\n");
    }

    return 0;
}
```

OUTPUT:

```
=== Smart Waste Recycling Plant System ===
Daily Target Waste Capacity: 5000 kg
Special Codes: Enter '-1' for Maintenance Day | '-2' for Machine Failure

--- Day 1 ---
Enter total recycled waste (kg): 4600
  Recycled Waste  : 4600.00 kg (92.00%)
  Remaining Waste : 400.00 kg
  Efficiency Level: EXCELLENT

--- Day 2 ---
Enter total recycled waste (kg): -1
-> Status: Plant Maintenance Day. Skipping record...

--- Day 3 ---
Enter total recycled waste (kg): 1800
  Recycled Waste  : 1800.00 kg (36.00%)
  Remaining Waste : 3200.00 kg
  Efficiency Level: POOR
```

20. Smart Metro Rail Passenger Management System
A metro rail corporation operates 12 stations with a maximum capacity of 2,000 passengers per station per hour.
Passenger Status:
≤1,000 → Low Crowd
1,001–1,500 → Moderate Crowd
1,501–2,000 → Heavy Crowd
Above 2,000 → Overcrowded
If overcrowding continues for 2 consecutive hours, additional trains should be deployed.
(a)Develop a C program using arithmetic operators to calculate the average number of passengers per hour. Use decision-making statements to classify the crowd level and determine whether additional trains are required.
(b) Use looping constructs to process passenger data for all metro stations. Use continue to skip stations under maintenance and break when metro services are suspended due to an emergency.

PSEUDO CODE:

START

Initialize total_passengers = 0, processed_stations = 0

FOR station = 1 TO 12 DO

PRINT "Station station: Enter Passenger Count for Hour 1 and Hour 2 (or -1 for Maintenance, -2 for Emergency):"

READ h1_count, h2_count

// Use continue to skip stations under maintenance

IF h1_count == -1 OR h2_count == -1 THEN

PRINT "Station station is under maintenance. Skipping..."

CONTINUE

END IF

// Use break when metro services are suspended due to emergency

IF h1_count == -2 OR h2_count == -2 THEN

PRINT "EMERGENCY ALERT: Metro services suspended! Halting data collection."

BREAK

END IF

// Calculate average passengers per hour for this station

station_avg = (h1_count + h2_count) / 2.0

// Accumulate total for system-wide metrics

```
        total_passengers = total_passengers + (h1_count + h2_count)
        processed_stations = processed_stations + 1

        // Classify crowd level based on 2-hour average
        IF station_avg <= 1000 THEN
            status = "Low Crowd"
        ELSE IF station_avg <= 1500 THEN
            status = "Moderate Crowd"
        ELSE IF station_avg <= 2000 THEN
            status = "Heavy Crowd"
        ELSE
            status = "Overcrowded"
        END IF

        PRINT "Average Passengers/Hour: station_avg"
        PRINT "Crowd Level: status"

        // Check if additional trains are required (overcrowded for 2 consecutive hours)
        IF h1_count > 2000 AND h2_count > 2000 THEN
            PRINT "ACTION REQUIRED: Deploy Additional Trains! (Overcrowded for 2
consecutive hours)"
        ELSE
            PRINT "ACTION: Normal Train Operations"
        END IF
    END FOR

    // Overall Summary
    IF processed_stations > 0 THEN
        overall_avg = total_passengers / (processed_stations * 2.0)
        PRINT "Processed Stations: processed_stations"
        PRINT "Overall System Average Passengers/Hour: overall_avg"
    END IF
```

END

C PROGRAM:

```
#include <stdio.h>

int main() {
    int total_stations = 12;
    int h1, h2;
    int total_passengers = 0;
    int processed_stations = 0;

    printf("=== Smart Metro Rail Passenger Management System ===\n");
    printf("Max Capacity: 2000 passengers/station/hour\n");
    printf("Special Codes: Enter '-1' for Maintenance | '-2' for Emergency Suspension\n\n");

    for (int station = 1; station <= total_stations; station++) {
        printf("--- Station %d ---\n", station);
        printf("Enter passenger counts for Hour 1 and Hour 2: ");
        if (scanf("%d %d", &h1, &h2) != 2) {
            printf("Invalid input format. Exiting...\n");
            break;
        }

        // (b) Skip station if under maintenance
        if (h1 == -1 || h2 == -1) {
            printf("  -> Station %d is under maintenance. Skipping...\n\n", station);
            continue;
        }

        // (b) Break loop if services are suspended due to an emergency
        if (h1 == -2 || h2 == -2) {
```

```
            printf("  -> EMERGENCY ALERT: Metro services suspended! Halting station monitoring.\n\n");
            break;
        }

        // (a) Calculate average passengers per hour using arithmetic operators
        float station_avg = (h1 + h2) / 2.0;
        total_passengers += (h1 + h2);
        processed_stations++;

        printf("  Average Hourly Volume: %.2f passengers/hr\n", station_avg);

        // (a) Classify crowd level using decision-making statements
        if (station_avg <= 1000) {
            printf("  Crowd Status        : Low Crowd\n");
        }
        else if (station_avg <= 1500) {
            printf("  Crowd Status        : Moderate Crowd\n");
        }
        else if (station_avg <= 2000) {
            printf("  Crowd Status        : Heavy Crowd\n");
        }
        else {
            printf("  Crowd Status        : Overcrowded\n");
        }

        // (a) Determine whether additional trains are required (2 consecutive hours > 2000)
        if (h1 > 2000 && h2 > 2000) {
            printf("  Action Triggered    : DEPLOY ADDITIONAL TRAINS (Consecutive overcrowding detected!)\n\n");
        } else {
            printf("  Action Triggered    : Normal Schedule\n\n");
        }
```

```
    }

    // Network Summary
    printf("=================================================\n");
    printf("              NETWORK SUMMARY                    \n");
    printf("=================================================\n");
    if (processed_stations > 0) {
        float overall_avg = (float)total_passengers / (processed_stations * 2);
        printf("Active Stations Monitored : %d\n", processed_stations);
        printf("Total Network Passengers  : %d\n", total_passengers);
        printf("Overall Hourly Average    : %.2f passengers/hr/station\n", overall_avg);
    } else {
        printf("No active station data processed.\n");
    }

    return 0;
}
```

OUTPUT:

```
=== Smart Metro Rail Passenger Management System ===
Max Capacity: 2000 passengers/station/hour
Special Codes: Enter '-1' for Maintenance | '-2' for Emergency Suspension

--- Station 1 ---
Enter passenger counts for Hour 1 and Hour 2: 800 950
  Average Hourly Volume: 875.00 passengers/hr
  Crowd Status         : Low Crowd
  Action Triggered     : Normal Schedule

--- Station 2 ---
Enter passenger counts for Hour 1 and Hour 2: -1 0
  -> Station 2 is under maintenance. Skipping...
```